


# National Institute of Technology Raipur

# Distributed Systems Lab File

**BY**

**Raghav Mahajan**

**RollNo. - 23115076**

**(7th Semester, Computer Science & Engineering)**

# INDEX

| S. No. | Date | Experiment/ Practical | Page No. | Signature |
|---|---|---|---|---|
| 1. | | Implement concurrent echo client-server application in JAVA | | |
| 2. | | Implement a Distributed Chat Server using TCP Sockets in JAVA | | |
| 3. | | Implement concurrent day-time client-server application in JAVA | | |
| 4. | | Configure following options on server socket and test them :SO_KEEPALIVE, SO_LINGER, SO_SNDBUF, SO_RCVBUF, TCP_NODELAY | | |
| 5. | | Write a program to Incrementing a counter in shared memory in JAVA | | |
| 6. | | Write a program to Simulate the Distributed Mutual Exclusion. | | |
| 7. | | Write a program to Implement Java RMI mechanism for accessing method | | |
| 8. | | Write a program to Create CORBA based server-client application | | |

# Practical 1

**Aim:** Implement Concurrent echo client-server application in java

**Theory:** Developing a concurrent client-server application in Java where multiple clients can connect to the server simultaneously. The server echoes back the messages sent by each client, demonstrating the use of multithreading and socket programming for parallel communication.

## CODE:

### ConcurrentEchoS er ver.java

```
import java.io.*;
import java.net.*;

public class ConcurrentEchoServer {
	public static void main(String[] args) {
		int port = 1234;
		try (ServerSocket serverSocket = new ServerSocket(port)) {
			System.out.println("Concurrent Echo Server listening on port " + port + "...");
			while (true) {
				Socket clientSocket = serverSocket.accept();
				System.out.println("New client connected: " + clientSocket.getInetAddress().g
				new Thread(new ClientHandler(clientSocket)).start();
			}
		} catch (IOException e) {
			System.out.println("Server exception: " + e.getMessage());
			e.printStackTrace();
		}
	}

	private static class ClientHandler implements Runnable {
		private Socket socket;
		public ClientHandler(Socket socket) {
			this.socket = socket;
		}

		@Override
		public void run() {
			try (
				BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputS
				PrintWriter out = new PrintWriter(socket.getOutputStream(), true)
			) {
				String message;
				while ((message = in.readLine()) != null) {
					System.out.println("Received: " + message);
					out.println(message);
				}
			} catch (IOException e) {
				System.out.println("Client disconnected unexpectedly.");
			} finally {
				try {
```

```
                socket.close();
            }  catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

### EchoClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

classEchoClient {
    public  static   void main(String[] args) {
        String  hostname = "127.0.0.1";
        int port= 1234;
        try  (
            Socket  socket = new Socket(hostname, port);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStrea
            Scanner scanner = new Scanner(System.in)
        )  {
            System.out.println("Connected to the Echo Server. Type 'exit' to quit.");
            while  (true) {
                System.out.print("Enter message: ");
                String userInput = scanner.nextLine();
                if   ("exit".equalsIgnoreCase(userInput)) {
                    break;
                }
                out.println(userInput);
                String serverResponse = in.readLine();
                System.out.println("Server Echo: " + serverResponse);
            }
        }  catch  (UnknownHostException e) {
            System.out.println("Server not found: " + e.getMessage());
        }  catch  (IOException e) {
            System.out.println("I/O error: " + e.getMessage());
        }
    }
}
```

## OUTPUT:

TERMINAL 1: SERVER

```
raghav_mahajan@fedora ~/Documents/Lab-DS/P1    $ java ConcurrentEchoServer
Concurrent Echo Server listening on port 1234... New
client connected: 127.0.0.1
Received: Hi! This is test message.
```

TERMINAL 2: CLIENT

raghav_mahajan@fedora ~/Documents/Lab-DS/P1

Connected to the Echo Server. Type 'exit' to quit.

Enter message: Hi! This is test message.

Server Echo: Hi! This is test message.

Enter message:

# Practical 2

*Aim :* Implement a distributed chat server using TCP sockets in Java.

*Theory :* A distributed chat application enables multiple clients to communicate over a network using a central server. The server utilizes TCP sockets to maintain reliable, bi-directional connections with each client. Multithreading is employed to handle concurrent client connections, enabling real-time message broadcasting across the network. A graphical user interface (GUI) built using Java Swing provides a seamless and interactive user experience.

CODE:

**ChatServer.java**

```
import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer {
	private static Set<PrintWriter> clientWriters = new HashSet<>();

	public static void main(String[] args) {
		System.out.println("GUI Chat Server listening on port 1234...");
		try (ServerSocket serverSocket = new ServerSocket(1234)) {
			while (true) {
				new ClientHandler(serverSocket.accept()).start();
			}
		} catch (Exception e) {
			e.printStackTrace();
		}
	}

	private static class ClientHandler extends Thread {
		private Socket socket;
		private PrintWriter out;
		private BufferedReader in;

		public ClientHandler(Socket socket) {
			this.socket = socket;
		}

		public void run() {
			try {
				in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
				out = new PrintWriter(socket.getOutputStream(), true);
				synchronized (clientWriters) {
					clientWriters.add(out);
```

```
                    }
                    System.out.println("New user connected.");

                    String message;
                    while ((message = in.readLine()) != null) {
                        synchronized (clientWriters) {
                            for (PrintWriter writer : clientWriters) {
                                writer.println(message);
                            }
                        }
                    }
                } catch (IOException e) {
                    System.out.println("User disconnected.");
                } finally {
                    if (out != null) {
                        synchronized (clientWriters) {
                            clientWriters.remove(out);
                        }
                    }
                    try {
                        socket.close();
                    } catch (IOException e) {
                        e.printStackTrace();
                    }
                }
            }
        }
    }
```

**ChatClientGUI.java**

```
import javax.swing.*;
import javax.swing.text.html.HTMLDocument;
import javax.swing.text.html.HTMLEditorKit;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;

public class ChatClientGUI {
    private BufferedReader in;
    private PrintWriter out; private
    JFrame frame;
    private JEditorPane chatPane;
    private JTextField textField;
    private String clientName;
    private HTMLEditorKit kit;
    private HTMLDocument doc;
```

```
    public ChatClientGUI() {
        clientName = JOptionPane.showInputDialog(
            null, "Enter your name:", "Name Selection",
JOptionPane.PLAIN_MESSAGE);
        if (clientName == null || clientName.trim().isEmpty()) {
            System.exit(0);
        }

        frame = new JFrame(clientName + "'s Chat");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 500);

        chatPane = new JEditorPane();
        chatPane.setEditable(false);
        chatPane.setContentType("text/html");
        kit = new HTMLEditorKit();
        doc = new HTMLDocument();
        chatPane.setEditorKit(kit);
        chatPane.setDocument(doc);

        frame.getContentPane().add(new JScrollPane(chatPane),
BorderLayout.CENTER);

        JPanel bottomPanel = new JPanel();
        bottomPanel.setLayout(new BorderLayout());
        textField = new JTextField();
        bottomPanel.add(textField, BorderLayout.CENTER);

        JButton sendButton = new JButton("Bhejo!");
        bottomPanel.add(sendButton, BorderLayout.EAST);
        frame.getContentPane().add(bottomPanel, BorderLayout.SOUTH);

            ActionListener sendListener = new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                    String message = textField.getText();
                    if (message != null && !message.trim().isEmpty()) {
                        out.println(clientName + ": " + message);
                        textField.setText("");
                    }
                }
            };
        textField.addActionListener(sendListener);
        sendButton.addActionListener(sendListener);
    }

    private void appendToPane(String message, String color) {
        try {
            String htmlText = "<span style='color:" + color + "; font-
family:sans-serif;'>" + message + "</span><br>";
            kit.insertHTML(doc, doc.getLength(), htmlText, 0, 0, null);
            chatPane.setCaretPosition(doc.getLength());
```

```
            } catch (Exception e) {
                e.printStackTrace();
            }
        }

        private voidrun() throws IOException {
            Socket socket = newSocket("127.0.0.1", 1234);
            in =newBufferedReader(new InputStreamReader(socket.getInputStream()));
            out = newPrintWriter(socket.getOutputStream(), true);

            while (true){
                String line = in.readLine();
                if (line== null) break;
                if (line.startsWith(clientName + ":")) {
                    appendToPane(line, "blue");
                } else {
                    appendToPane(line, "black");
                }
            }
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(new Runnable() {
                public void run() {
                    ChatClientGUI client = new ChatClientGUI();
                    client.frame.setVisible(true);
                    newThread(newRunnable() {
                        public void run() {
                            try {
                                client.run();
                            } catch (IOException e) {
                                e.printStackTrace();
                            }
                        }
                    }).start();
                }
            });
        }
    }
```

OUTPUT:

TERMINAL 1: SERVER

```
┌──raghav_mahajan@fedora ~/Documents/Lab-DS/P2
└─$ java ChatServer
GUI Chat Server listening on port 1234...
New user connected.
New user connected.
New user connected.
```

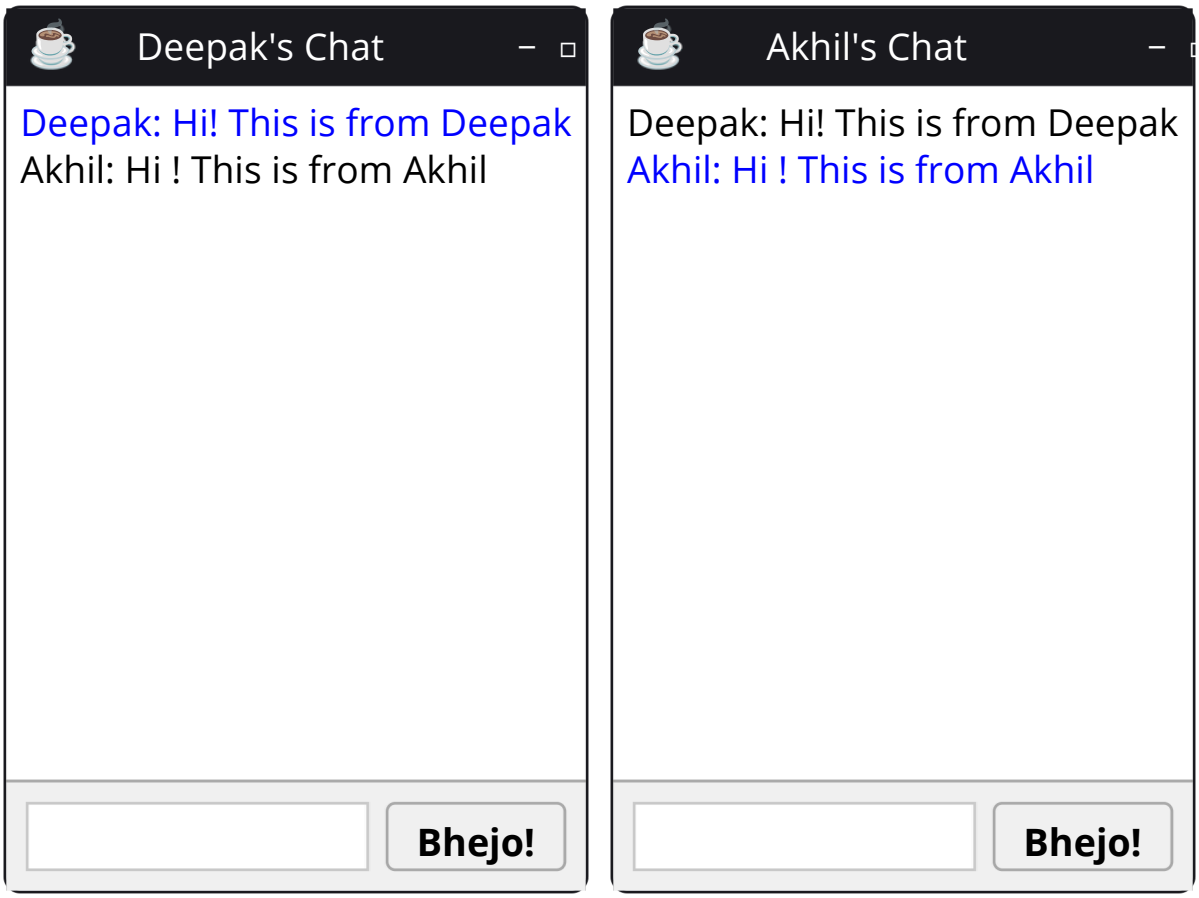
TERMINAL 2: CLIENT 1 (DEEPAK)

```
┌──raghav_mahajan@fedora ~/Documents/Lab-DS/P2
└─$ java ChatClientGUI
```

TERMINAL 3: CLIENT 2 (AKHIL)

```
┌──raghav_mahajan@fedora ~/Documents/Lab-DS/P2
└─$ java ChatClientGUI
```

## GUI OUTPUT:



# Practical 3

**Aim:** Implementation of concurrent day-time client server application.

**Theory:** A day-time server provides the current date and time to any client that connects to it. In a concurrent implementation, the server is capable of handling multiple client requests simultaneously by creating a new thread for each incoming TCP connection. This ensures that the server remains responsive and does not block while servicing a particular client, demonstrating effective use of Java Socket programming and multithreading.

## CODE:

### Server_DT.java

```
import java.io.*;
import java.net.*;
import java.util.Date;

public class Server_DT {
	public static void main(String[] args) {
		int port = 5000;
		try (ServerSocket serverSocket = new ServerSocket(port)) {
			System.out.println("The Server has reserved port No.: " + port + " for this Servi
			while (true) {
				Socket clientSocket = serverSocket.accept();
				System.out.println("Client with IP Address " + clientSocket.getInetAddress()
				new Thread(new ClientHandler(clientSocket)).start();
			}
		} catch (IOException e) {
			e.printStackTrace();
		}
	}

	private static class ClientHandler implements Runnable {
		private Socket socket;

		public ClientHandler(Socket socket) {
			this.socket = socket;
		}

		@Override
		public void run() {
			try (PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {
				out.println(new Date().toString());
			} catch (IOException e) {
				e.printStackTrace();
			} finally {
				try {
					socket.close();
				} catch (IOException e) {
					e.printStackTrace();
				}
```

```
                }
            }
        }
    }
```

### Client_DT.java

```
import java.io.*;
import java.net.*;

public  classClient_DT {
      public  static   void main(String[] args) {
            String  hostname = "127.0.0.1";
            int port= 5000;
            try  (Socket  socket = new Socket(hostname, port)) {
                  System.out.println("Client     localhost/"     +     socket.getLocalAddress().getHostAddress(
                  BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStrea
                  String  serverTime = in.readLine();
                  System.out.println("Current Date & Time on Server is: " + serverTime);
            }  catch  (UnknownHostException e) {
                  e.printStackTrace();
            }  catch  (IOException e) {
                  e.printStackTrace();
            }
      }
}
```

## OUTPUT:

TERMINAL 1: SERVER

raghav_mahajan@fedora ~/Documents/Lab-DS/P3

The Server has reserved port No.: 5000 for this Service

Client with IP Address /127.0.0.1 has communicated via port No. 41450

TERMINAL 2: CLIENT

raghav_mahajan@fedora ~/Documents/Lab-DS/P3

$ java Client_DT Client localhost/127.0.0.1 is communicating from port No.: 41450 Current Date & Time on Server is: Tue Aug 18 12:20:52 IST 2026

# Practical 4

**Aim:** Configure following options on server socket and tests them: SO_KEEPALIVE, SO_LINGER, SO_SNDBUF, SO_RCVBUF, TCP_NODELAY.

**Theory:** Socket options in Java allow developers to fine-tune the behavior of the underlying network protocols (like TCP) to optimize performance, handle edge cases, or adjust latency. SO_KEEPALIVE ensures inactive connections are checked periodically. SO_LINGER specifies how long to wait before forcefully closing a socket if un-sent data remains. SO_SNDBUF and SO_RCVBUF set hints for the size of the underlying network I/O buffers. Finally, TCP_NODELAY disables Nagle's algorithm, forcing packets to be sent immediately rather than waiting to fill a buffer, which is crucial for time-sensitive or low-latency applications.

## CODE:

### SocketOptionsTest.java

```
import java.net.*;
import java.io.*;

public  classSocketOptionsTest {
      public  static void main(String[] args) {
            try(ServerSocket serverSocket = new ServerSocket(5000)) {
               System.out.println("ServerSocket SO_RCVBUF: " + serverSocket.getReceiveBufferSize
               System.out.println("Waiting for a client connection on port 5000...");
               Socket clientSocket = serverSocket.accept();

               clientSocket.setKeepAlive(true);
               clientSocket.setSoLinger(true, 50);
               clientSocket.setSendBufferSize(8192);
               clientSocket.setReceiveBufferSize(8192);
               clientSocket.setTcpNoDelay(true);

               System.out.println("--- Client Socket Options ---");
               System.out.println("SO_KEEPALIVE: " + clientSocket.getKeepAlive());
               System.out.println("SO_LINGER: " + clientSocket.getSoLinger() + " seconds");
               System.out.println("SO_SNDBUF: " + clientSocket.getSendBufferSize() + " bytes");
               System.out.println("SO_RCVBUF: " + clientSocket.getReceiveBufferSize() + " bytes"
               System.out.println("TCP_NODELAY: " + clientSocket.getTcpNoDelay());

               clientSocket.close();
               catch (IOException e) {
            }
                  e.printStackTrace();
            }
      }
}
```

## OUTPUT:

```
TERMINAL 1: SERVER
raghav_mahajan@fedora~/Documents/Lab-DS/P4
ServerSocket SO_RCVBUF: 65536
Waiting for a client connection on port 5000... ---
Client Socket Options ---
SO_KEEPALIVE: true
SO_LINGER: 50 seconds
SO_SNDBUF: 8192 bytes
SO_RCVBUF: 8192 bytes
TCP_NODELAY: true
```

```
TERMINAL 2: CLIENT
raghav_mahajan@fedora ~/Documents/Lab-DS/P4  $ telnet localhost 5000
Trying ::1...
Connected to localhost.
Escape character is '^]'.
Connection closed by foreign host.
raghav_mahajan@fedora ~/Documents/Lab-DS/P4  $
```